LLM 量化四篇奠基论文学习笔记

阅读对象： LLM.int8()、SmoothQuant、GPTQ、AWQ **适用阶段：** 已通读一轮，准备建立方法之间的联系并做复现实验 **整理日期：** 2026-10-08 **页码口径：** 下文“PDF 第 n 页”指用户提供文件从封面开始计数的物理页码。

先看全局：四篇分别回答什么问题

论文	主要目标	核心动作	最常见误读
LLM.int8()	在大模型上保住 INT8 推理精度	向量级量化，并把异常特征维度交给 FP16 路径	“8-bit”不等于所有计算都用 INT8，也不保证更快
SmoothQuant	让权重和激活都能做 INT8 计算，即 W8A8	等价缩放，将部分激活量化难度转移给权重	不是简单删去或裁剪激活异常值
GPTQ	用 3/4-bit 权重保持较好精度	根据校准输入的二阶信息，量化一列后调整未量化权重	Hessian 感知不等于微调，也不等于额外 LoRA 分支
AWQ	让低位仅权重量化更准、更易部署	用激活识别重要通道，缩放后再量化权重	“保留 1% FP16”是动机实验，不是最终部署格式

一个便于记忆的动词链：**LLM.int8() 隔离异常值 → SmoothQuant 迁移异常值影响 → GPTQ 顺序补偿舍入误差 → AWQ 缩放保护重要权重通道。**这只是教学归纳，不是论文给出的统一分类。

共同记号与评估边界

下面统一用行样本约定：X 是 [样本或 token 数, 输入维度]，W 是 [输入维度, 输出维度]，线性层输出为 Y = XW。量化与反量化后的浮点等效矩阵记为 Wq。GPTQ 原文把样本放在列上，因此其式子写作 WX；阅读时先核对矩阵维度，避免把转置差异误认成方法差异。

一个对称均匀量化器可理解为 Q(z) = clip(round(z / step), qmin, qmax) * step。step 越大，表示范围越宽而网格越粗；因此极少数大值可能让多数普通值只占很少的有效格点。位宽、scale 的统计范围、静态或动态标定，都影响误差。

W8A8 表示权重与激活均按 8-bit 路径处理；W4A16 表示权重低位、激活保持高精度。**PPL 越低越好**，但只有模型、数据、分词、序列长度等设置一致时，数字才适合直接比较。论文宣称的加速还依赖内核、硬件、批大小与 prefill/decode 阶段。

1. LLM.int8()：先认出并隔离异常特征

原文： Dettmers 等，LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale，NeurIPS 2022；本地 PDF 20 页，arXiv:2208.07339v2。

研究问题与关键观察

如果整张激活矩阵只用一个最大绝对值来定量化 scale，少数极大值会拉大步长，多数普通值的舍入误差随之增大。作者发现，在其研究的大模型中，异常激活并非随机散落，而是集中于很少的**特征维度**；随着模型规模增大，这些维度会持续影响许多层与 token。由此，单纯提高每张量量化精度仍可能不够。（§1、§3，PDF 第 1–5 页）

方法拆成两个相连的步骤

1. **向量级量化。** 把矩阵乘法视作多次内积：对 X 的每一行、W 的每一列分别设 scale；INT8 内积以 INT32 累积，再用行 scale 与列 scale 的外积还原输出。它比全张量只用一组 scale 更细。注意：作者发现只做这一步仍难处理较大模型里的固定异常特征维度。（§3.1，式 (7)，PDF 第 4 页）
2. **混合精度分解。** 找出激活绝对值超过阈值的特征维度，将这些维度对应的 X 与 W 拿到 FP16 路径相乘；其余维度用 INT8，最后合并输出。原文实验取阈值 6.0，这是该论文设置，不应当当作所有模型的通用常数。（§3.2，式 (8)、图 2，PDF 第 3、5 页）

可把前向过程记作：Y ≈ INT8_path(X_regular, W_regular) + FP16_path(X_outlier, W_outlier)。这里隔离的是少数**维度及相应矩阵块**，不是在最终权重里随机挑几个数保持 FP16。

看一组证据，同时看清边界

在论文的 C4 验证集与 13B 模型设置下，表 1 报告：32-bit 基线 PPL 为 **12.45**，仅用 absmax 向量级 INT8 为 **16.48**，完整 LLM.int8() 为 **12.45**。这支持“向量级 scale 加异常维度分解可以恢复该设置的精度”，不能外推为所有模型都完全无损。（表 1，PDF 第 6 页）

性能目标首先是**显存与精度**。作者在讨论中明确没有量化注意力函数本身，也没有解决大规模训练；附录的 BLOOM-176B 端到端推理表显示 INT8 在相同 8×A100 配置下略慢于 bfloat16（batch 1：253 与 239 ms/token），但可以用更少 GPU 跑接近的延迟。（§6，PDF 第 10 页；附录表 6，PDF 第 18 页）

读完要能回答

- 为什么异常值若集中在输入特征列，按 token 行设置 scale 仍可能无效?
- 把异常维度单独处理之后，INT8 路径与 FP16 路径的矩阵形状各是什么?
- 论文里的“几乎没有精度损失”对应什么数据集、模型规模与指标?

一句话记忆： 大多数值使用便宜的 INT8；少数系统性异常特征维度使用 FP16，换取较稳定的预测质量。

2. SmoothQuant：保持函数不变，重新分配量化难度

原文： Xiao 等，SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models，ICML 2023；本地 PDF 13 页，arXiv:2211.10438v7。

为什么需要新的处理

论文观察到大激活值倾向于持续出现在固定输入通道；激活按通道量化虽然可能保精度，但在其 INT8 GEMM 路径中，输入通道是矩阵乘的归约维度，逐通道 scale 不易像 token 或输出通道的 scale 那样放到乘法之后处理。表 1 中，OPT-175B 的四任务平均准确率为 FP16 **71.6%**、INT8 per-token **31.7%**、模拟的 INT8 per-channel **71.4%**。这说明量化粒度与硬件映射之间存在张力。（§3，图 3、表 1，PDF 第 3 页）

关键等价式：先变换，再量化

对每个输入通道选一个正的缩放因子 s_j，令 S = diag(s)：

XW = (X S^-1)(S W)。

在**尚未量化**时，两侧严格相等：激活除以 s_j，权重对应的输入通道乘以 s_j。若 s_j 较大，原本突出的激活通道会变平，但相应权重会变大。因此目标是分担难度，而不是让异常信息消失。（§4，式 (3)，PDF 第 4 页）

原文给出的选择是 s_j = max(|X_j|)^alpha / max(|W_j|)^(1-alpha)。alpha 越大，向权重转移越多量化难度。论文对 OPT、BLOOM 常用 alpha = 0.5，对较难量化的 GLM-130B 用更大的值；它同时在校准集上确定激活统计量。模型经等价变换后，才对 X S^-1 和 S W 做 INT8 量化，所以**变换前后函数严格等价**与**量化后误差为零**是两回事。（§4，式 (4)；§5.1，PDF 第 4–5 页）

O1/O2/O3 如何读

论文给出由精度较细到实现更快的三档：O1 使用 per-token 动态激活量化；O2 使用 per-tensor 动态激活量化；O3 使用 per-tensor 静态激活量化，权重在表 2 中均按 per-tensor 设置。它们共享平滑变换，差别主要在**变换之后采用哪种量化策略**。论文的 INT8 加速声明还依赖可用的 INT8 GEMM 实现。（表 2，PDF 第 5 页）

实验读法与限制

OPT-175B 的七项 zero-shot 平均准确率：FP16 **66.9%**，SmoothQuant-O3 **66.8%**；WikiText PPL 则从 **10.99** 到 **11.17**。在论文的 PyTorch 实现中，图 8 报告最高约 **1.51 倍**速度提升和 **1.96 倍**峰值显存节省；FasterTransformer 实现报告最高约 **1.56 倍**。这些是特定模型、硬件及内核下的结果。（表 3，PDF 第 6 页；图 8，PDF 第 8 页）

这里的 W8A8 指计算密集型线性层与注意力矩阵乘采用 INT8 路径；Softmax、LayerNorm 等轻量操作仍可保留 FP16。不要把 W8A8 解读为网络里的**每个算子**都使用 INT8。（图 6，PDF 第 5 页）

读完要能回答

- 为什么 XW = (XS^-1)(SW) 在浮点下严格成立，但量化之后未必等价？
- 当 alpha 太大，为什么激活容易量化、权重却可能变难？
- 激活 per-channel scale 为什么在论文考虑的 INT8 GEMM 路径里不方便？

一句话记忆： 用离线可融合的等价缩放，让原本难量化的激活和容易量化的权重共同承担误差，从而使用 W8A8 内核。

3. GPTQ：用二阶信息逐步修正权重量化误差

原文： Frantar 等，GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers，ICLR 2023；本地 PDF 16 页，arXiv:2210.17323v2。

优化目标先于算法

GPTQ 做的是逐层、仅权重的 PTQ。对校准输入 X，目标是找到低位权重 Wq，使层输出误差 ||WX - WqX||_F^2 小，而非只最小化 ||W-Wq||_F^2。此处沿用论文把样本放在列上的约定。对一行权重，这个平方目标是二次型，Hessian 与输入的 Gram 矩阵 2XX^T 有关：输入各维出现频率与相关性会改变“某个权重舍入错一点”对输出的影响。（§3，式 (1)–(2)，PDF 第 3–4 页）

式 (2) 的直觉

假设现在把第 q 个权重从 w_q 舍入到网格点 Q(w_q)。如果只改它，误差完全由这一步承担；GPTQ 则沿着逆 Hessian 给出的相关方向，调整尚未量化的权重 w_F，抵消一部分输出变化。原文的更新含有 H^-1 的第 q 列和对角元 H^-1_qq。这属于**离线量化时改写剩余权重**，部署时没有必然新增一条 LoRA 式低秩计算分支。（§3，式 (2)，PDF 第 4 页）

为什么它能扩到大模型

GPTQ 从前身 OBQ 借用二阶补偿，但做了三项关键工程变化：

1. **同一固定量化顺序。** 不为每行权重分别搜索贪心顺序，使 Hessian 信息可以跨输出行共享。（§4 Step 1，PDF 第 4 页）
2. **懒更新与块处理。** 块内逐列量化，将对远处列的更新合并成矩阵运算；论文示例块大小 B=128，主要改善 GPU 的访存与运算利用率。（§4 Step 2，PDF 第 5 页）
3. **Cholesky 重写与阻尼。** 稳定地使用逆 Hessian 信息，避免反复更新矩阵导致数值不稳定；算法 1 写出完整过程。（§4 Step 3、算法 1，PDF 第 5 页）

因此，“二阶”在这里指**用输入的二阶统计指导舍入误差补偿**，不意味着重新训练整个模型。

实验读法与限制

论文在 C4 上取 **128 段、每段 2048 token** 用于校准，并以单张 A100 80GB 逐块量化模型。在 OPT-175B 的 WikiText2 测试上，FP16、4-bit RTN、4-bit GPTQ 的 PPL 分别为 **8.34、10.54、8.37**；完整 OPT-175B 的 GPTQ 量化时间为 **4.2 小时**。这些数字分别来自表 3、表 2，不能忽略位宽、校准与机器设置。（§5、表 2–3，PDF 第 6–7 页）

其 3-bit OPT-175B 的 batch 1 生成实验中，A100 上平均每 token 延迟由 FP16 的 **230 ms** 降为 GPTQ 的 **71 ms**，但同时 GPU 数从 **5 张**降至 **1 张**；加速来自低位权重加载和专门的反量化矩阵向量内核，论文明确没有声称低位乘法本身减少了计算量。（表 6，PDF 第 8 页；§6，PDF 第 9 页）

读完要能回答

- H = 2XX^T 中为何不需要反向传播求整模型 Hessian?
- 若一个权重已经被量化，为什么只调整尚未量化的权重?
- GPTQ 与“给已量化矩阵再加低秩补偿”在参数存放和推理图上有何区别?

一句话记忆： 对一列权重舍入时，用校准输入的二阶统计指导剩余权重的更新，让低位权重尽量保住层输出。

4. AWQ：用激活找重要通道，再缩放保护权重

原文： Lin 等，AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration，MLSys 2024；本地 PDF 15 页，arXiv:2306.00978v6。

激活感知如何进入仅权重量化

AWQ 的目标是 W3A16 / W4A16 一类低位**仅权重**量化。作者先做了一个诊断实验：在 INT3、group size 128 的 OPT-6.7B 上，RTN 的 WikiText PPL 为 **23.54**；若根据激活幅度选 1% 的重要权重通道保留 FP16，PPL 为 **11.39**；按权重大小选 1% 则是 **22.37**。这个实验支持“激活幅度有助于定位敏感通道”，不意味着最终 AWQ 模型真的把 1% 权重留作 FP16。（§3.1、表 1，PDF 第 3–4 页）

从动机实验走向实际算法

使用与前文相同的行样本记号，AWQ 可表示为 XW = (X S^-1)(S W)：先放大重要输入通道对应的权重，再对输入施加反向缩放。原论文写作 WX = (W diag(s))(diag(s)^-1 X)，差别只是矩阵摆放约定。全精度函数不变，但量化网格上的舍入行为会变。若放大某通道的因子为 s>1，且所属量化组的步长 step_new ≈ step_old，该通道经反向缩放后的误差可近似降为原来的 1/s；若步长也随之增大，其他通道会受损，所以不能无限放大。（§3.2，式 (1)–(3)、表 2，PDF 第 4 页）

AWQ 把每个输入通道的平均激活幅度记作 a_j，取候选缩放 s_j = a_j^alpha，通过校准输入上的层输出差异网格搜索 alpha，并进一步对权重做 clipping。作者在实验中使用 20 个候选网格点。因此“无需反向传播或回归重建”**不等于完全不看输出误差**：其超参数搜索仍以量化前后的层输出差异为依据。（§3.2，式 (4)–(5)，PDF 第 5 页；§5.1，PDF 第 7 页）

精度和加速需要分开归因

在论文的 Llama-2-7B、INT4-g128、WikiText2 设置下，FP16 PPL **5.47**，RTN **5.73**，GPTQ **5.69**，AWQ **5.60**。这是特定实验设置的对照，不表示 AWQ 在所有模型或所有 GPTQ 配置上都更好。（表 4，PDF 第 7 页）

论文图 8 的 OPT-6.7B、INT3-g128 跨 PubMed / Enron 校准与评估实验显示，AWQ 在该设置下比 GPTQ 对校准分布变化更稳；这属于一项具体的分布偏移实验，不能概括为 GPTQ 普遍“过拟合”。（§5.3、图 8，PDF 第 10 页）

速度结果属于 **AWQ + TinyChat 系统**：权重打包、在线反量化融合、算子融合都参与了性能提升。图 9 还给出 TinyChat 自身 FP16 基线，说明部分速度收益来自系统实现。将“AWQ 提高精度”和“TinyChat 实现加速”分开记录。（§4、图 9，PDF 第 5–6、10–11 页）

读完要能回答

- 为什么查看权重绝对值，未必能找出量化后最影响输出的通道？
- AWQ 与 SmoothQuant 都有等价缩放；两者的量化对象和优化目标有何不同？
- 为什么放大重要通道过多可能伤害不重要通道？

一句话记忆： 用激活识别值得保护的权重通道，以等价缩放改变舍入误差分布，再用低位权重内核部署。

横向比较：放到同一张实验设计表里

对照维度	LLM.int8()	SmoothQuant	GPTQ	AWQ
量化路径	大部分 W8A8，异常维度 FP16	W8A8	主要 W3/4A16	主要 W3/4A16
关键数据	推理时的异常特征	离线激活通道最大值	校准输入的 Gram / 逆 Hessian	离线激活通道平均幅度
核心操作	分解矩阵乘路径	离线等价缩放	顺序量化并更新剩余权重	缩放、搜索 alpha、裁剪
额外约束	FP16 异常路径有开销	权重也不能被放大得太难量化	量化过程需计算二阶统计	缩放过大会伤及非重要通道
推理加速证据	不保证端到端提速	依赖 INT8 GEMM 实现	依赖低位权重加载内核	依赖 TinyChat 等部署系统

三组容易混淆的关系

LLM.int8() 对 SmoothQuant： 两者都关注大激活值。前者保留少数高精度路径；后者调整激活和权重分布，争取让计算密集型运算统一走 INT8。不能只看位宽相同就认为硬件代价相同。（LLM.int8() 图 2，PDF 第 3 页；SmoothQuant 图 6，PDF 第 5 页）

SmoothQuant 对 AWQ： 两者的浮点等价式外形相似，但 SmoothQuant 优先解决激活异常值带来的 W8A8 困难；AWQ 在 W3/4A16 下用激活信息保护被量化的重要权重通道。相同的代数变换服务于不同目标。（SmoothQuant §4，PDF 第 4 页；AWQ §3.2，PDF 第 4–5 页）

GPTQ 对后续 QERA / EoRA： GPTQ 在量化过程中更新未量化权重；后续低秩误差补偿会固定一个量化后的主体，另外构造低秩项。它们都关心输入如何放大量化误差，但变量、推理结构和额外代价不同。这里是为下一阶段学习提供的比较视角，不是四篇论文共同证明的结论。

一次小实验，把四篇串起来

在本机先用一个随机线性层：X 有 128 个输入维度，其中 4 个通道故意具有较大幅度；W 是 128×64。固定一份校准输入和另一份测试输入。

1. 对 W 做 4-bit RTN，记录 ||W-Wq||_F 与测试集 ||X_test(W-Wq)||_F。
2. 改用不同量化粒度，观察是否只降低权重误差，还是也降低输出误差。
3. 对 X 和 W 做 SmoothQuant 式等价缩放，先验证浮点输出不变，再分别量化，搜索 alpha。
4. 做 AWQ 式 s_j = mean(|X_calib,j|)^alpha 搜索，只量化权重，在相同测试输入上比较输出误差。
5. 若继续实现 GPTQ，先做单层、小维度、加阻尼的版本；核对量化前后的输出误差，再移植到更大模型。

第 3–4 步是**教学近似实验**，不是论文完整复现；也不能从 NumPy 浮点模拟的误差，推出 4-bit 实际存储或加速。对四种方法做模型级比较时，需统一模型、校准集、测试集、位宽、group size、序列长度、内核和硬件。

自测：能答出这些，再进入 QERA / EoRA

1. W8A8 和 W4A16 节省的资源、可能加速的阶段为何不同？
2. 为什么 PPL 较小只说明该评测设置下语言建模指标较好，而不直接证明推理更快？

3. LLM.int8() 的“向量级量化”和“混合精度分解”各解决什么缺口?
4. SmoothQuant 在浮点下严格等价，为什么选择 alpha 仍需校准?
5. GPTQ 的 Hessian 来源是什么？它在模型推理时是否还要重新计算?
6. AWQ 为什么先研究“1% FP16”，最终却采用缩放后的统一低位格式?
7. 比较 GPTQ 和 AWQ 时，为什么必须写清“位宽、group size、校准集、模型、评测集”？

建议把答不稳的题回到文中对应图和公式，不必重新通读全文。下一阶段读 QERA / EoRA 时，带着同一个问题：**权重差异变小与实际输入上的输出差异变小，是否总是同一回事?**

来源与核对方式

本笔记依据用户提供的四份 PDF，正文采用 **PDF 物理页码。**下列原论文链接用于检索；如果网上版本更新，具体表格数字以本笔记所用本地版本为准。

- Dettmers 等，LLM.int8()，本地文件 Dettmers-2022-LLM.int8_arXiv2208.07339.pdf；https://arxiv.org/abs/2208.07339
- Xiao 等，SmoothQuant，本地文件 Xiao-2022-SmoothQuant_arXiv2211.10438.pdf；https://arxiv.org/abs/2211.10438
- Frantar 等，GPTQ，本地文件 Frantar-2022-GPTQ_arXiv2210.17323.pdf；https://arxiv.org/abs/2210.17323
- Lin 等，AWQ，本地文件 AWQ ACTIVATION-AWARE WEIGHTQUANTIZATION FORON-DEVICE LLM COMPRESSION AND ACCELERATION.pdf；https://arxiv.org/abs/2306.00978